JAVA COLLECTIONS FRAMEWORK

Day 2 — Queue, Set & Map Classes

Interview-Ready Notes · Internal Working · Code · Mock Interview

Queue / PriorityQueue | HashSet | LinkedHashSet | TreeSet | HashMap | TreeMap | Hashtable

WHAT'S INSIDE

- Internal working of every class
- Real-world examples
- Complete runnable code
- 35–40 interview questions
- Coding problems with solutions
- AI-style interview questions
- Full mock interview transcript
- Hashing, load factor & internals deep dive

Prepared as a self-study & revision guide · Java SE

Table of Contents

Theory — Queue & PriorityQueue

1. Queue Interface Overview

`Queue` extends `Collection` and models a **FIFO (First-In-First-Out)** data structure, though some implementations (like PriorityQueue) reorder elements by priority rather than insertion time. `Deque` (double-ended queue) extends Queue and allows insertion/removal at both ends.

Queue Method	Behavior on failure	Purpose
`add(e)` / `offer(e)`	throws / returns false	Insert element
`remove()` / `poll()`	throws / returns null	Remove & return head
`element()` / `peek()`	throws / returns null	Return head without removing

Internal Working

Queue is an interface — the actual storage strategy depends on the implementation: `LinkedList` uses doubly linked nodes (true FIFO order), `ArrayDeque` uses a resizable circular array (faster than LinkedList for most queue/stack use, no node overhead), and `PriorityQueue` uses a **binary heap** array where the "head" is always the smallest (or highest-priority) element rather than the oldest one.

Real-world Example

A **print queue** or a **customer support ticket queue** — first ticket raised is (usually) the first one handled, modeled naturally as FIFO.

2. PriorityQueue

`PriorityQueue` is a Queue implementation where elements are ordered by **natural ordering** (Comparable) or a supplied `Comparator`, not insertion order. The head is always the smallest element (min-heap by default).

Internal Working

Internally it's a **binary min-heap** stored in a plain resizable array. The heap property guarantees `array[parent] <= array[child]` for every node. `offer()` appends the new element at the end of the array and then "bubbles it up" (`siftUp`) by swapping with its parent until the heap property is restored — O(log n). `poll()` removes the root (index 0, the minimum), moves the *last* element into the root position, and "bubbles it down" (`siftDown`) — also O(log n). `peek()` is O(1) since the minimum is always at index 0. Note: iterating a PriorityQueue with a plain for-each does **not** yield sorted order — only repeated poll() calls do.

Real-world Example

A **hospital emergency room triage system** — patients are treated by severity of condition (priority), not by arrival order. Also used in Dijkstra's shortest path algorithm and CPU task schedulers.

Complete Code Example

```
import java.util.PriorityQueue;
import java.util.Comparator;

public class PriorityQueueDemo {
    public static void main(String[] args) {
        // Min-heap (default) - smallest number first
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.offer(50);
        pq.offer(10);
        pq.offer(30);

        System.out.println("Peek (min): " + pq.peek());   // 10

        while (!pq.isEmpty()) {
            System.out.println("Poll: " + pq.poll());       // 10, 30, 50
        }

        // Max-heap using a custom Comparator
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
        maxHeap.offer(50);
        maxHeap.offer(10);
        maxHeap.offer(30);
        System.out.println("Max first: " + maxHeap.poll());  // 50
    }
}
```

Operation	Time Complexity
offer() / add()	O(log n)
poll() / remove()	O(log n)
peek()	O(1)
contains()	O(n)

Set Interface

3. Set Interface Overview

`Set` extends `Collection` and models a mathematical set: **no duplicate elements** allowed, with uniqueness determined by `equals()` / `hashCode()`. Unlike List, Set doesn't provide index-based access.

Implementation	Ordering	Backed by
HashSet	No guaranteed order	HashMap internally
LinkedHashSet	Insertion order preserved	LinkedHashMap internally
TreeSet	Sorted (natural or Comparator)	Red-Black Tree (TreeMap internally)

4. HashSet

Internal Working

`HashSet<E>` is literally a thin wrapper around a `HashMap<E, Object>` — every element you add becomes a **key** in that backing map, mapped to a shared dummy constant value called `PRESENT`. So `add(e)` internally calls `map.put(e, PRESENT)`, and uniqueness is enforced exactly the way HashMap enforces unique keys: via `hashCode()` to locate a bucket and `equals()` to check for collisions within that bucket. Because there's no linked list or tree tracking insertion order (beyond what the bucket layout happens to produce), iteration order is unpredictable and can change as the set grows.

Real-world Example

Tracking **unique visitor IDs** to a website in a day, where you only care "have I seen this ID already?" and don't care about order.

Complete Code Example

```
import java.util.HashSet;
import java.util.Set;

public class HashSetDemo {
    public static void main(String[] args) {
        Set<String> visitorIds = new HashSet<>();

        visitorIds.add("user_101");
        visitorIds.add("user_204");
        visitorIds.add("user_101");   // duplicate - ignored

        System.out.println(visitorIds.size());          // 2
        System.out.println(visitorIds.contains("user_101"));  // O(1) average
    }
}
```

5. LinkedHashSet

Internal Working

`LinkedHashSet` extends `HashSet` but is backed by a `LinkedHashMap` instead of a plain HashMap. LinkedHashMap augments every hash bucket entry with extra `before` / `after` pointers forming a doubly linked list across *all* entries in insertion order — so iteration order matches insertion order, at the cost of slightly higher memory per entry than HashSet.

Real-world Example

A **"recently used unique tags"** feature (e.g. a text editor's recent-unique-file list) where duplicates must be ignored but the order tags were first added still matters for display.

Complete Code Example

```
import java.util.LinkedHashSet;
import java.util.Set;

public class LinkedHashSetDemo {
    public static void main(String[] args) {
        Set<String> tags = new LinkedHashSet<>();
        tags.add("java");
        tags.add("collections");
        tags.add("java");     // duplicate ignored, order unaffected

        System.out.println(tags);   // [java, collections]  (insertion order kept)
    }
}
```

6. TreeSet

Internal Working

`TreeSet` is backed by a `TreeMap`, which implements a **Red-Black Tree** (a self-balancing binary search tree). Elements are kept sorted at all times — either by natural ordering (the element must implement `Comparable`) or by a `Comparator` passed to the constructor. Because the tree stays balanced, `add()`, `remove()`, and `contains()` are all **O(log n)** — slower than HashSet's O(1) average, but you gain sorted iteration and range operations like `first()`, `last()`, `headSet()`, `tailSet()`, and `ceiling()` / `floor()`.

Real-world Example

A **leaderboard of unique high scores** that must always be displayed sorted, or a scheduling system that needs "find the next available slot at or after time X" via `ceiling()`.

Complete Code Example

```
import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> scores = new TreeSet<>();
        scores.add(88);
        scores.add(45);
        scores.add(92);
        scores.add(67);

        System.out.println(scores);          // [45, 67, 88, 92] - always sorted
        System.out.println(scores.first());    // 45
        System.out.println(scores.last());     // 92
        System.out.println(scores.ceiling(70)); // 88 (smallest element >= 70)
        System.out.println(scores.floor(70));   // 67 (largest element <= 70)
    }
}
```

Feature	HashSet	LinkedHashSet	TreeSet
Order	Unspecified	Insertion order	Sorted order
add/remove/contains	O(1) average	O(1) average	O(log n)
Backed by	HashMap	LinkedHashMap	TreeMap (Red-Black Tree)
Allows null	One null	One null	No (throws NPE on natural ordering)

Map Interface

7. Map Interface Overview

`Map<K,V>` stores **key-value pairs** with unique keys. It is **not** a Collection (different hierarchy), though its view methods return real Collections.

Method	Purpose
`put(k, v)`	Insert or update a mapping
`get(k)`	Retrieve value for a key (null if absent)
`getOrDefault(k, def)`	Retrieve with a fallback default
`remove(k)`	Delete a mapping
`containsKey(k)` / `containsValue(v)`	Existence checks
`keySet()` / `values()` / `entrySet()`	Collection views for iteration
`merge()` / `compute()` / `putIfAbsent()`	Java 8+ atomic-style update helpers

8. HashMap

Internal Working (interview favorite — know this cold)

HashMap stores entries in an array of **buckets** (`Node<K,V>[] table`), default size **16**. For each `put(key, value)` : (1) Java computes `hash(key)` — the key's `hashCode()` is XOR'd with itself right-shifted 16 bits, to spread high-order bits into the lower bits and reduce collisions; (2) the bucket index is `hash & (table.length - 1)` , an efficient equivalent of modulo for power-of-two table sizes; (3) if the bucket is empty, a new `Node` is placed there; if not, HashMap walks the chain, using `.equals()` to check for an existing matching key (update) or appends a new node (collision). Since **Java 8**, if a single bucket's chain grows to **8 or more** nodes (and the table has at least 64 buckets), that bucket converts from a linked list to a **Red-Black Tree**, improving worst-case lookup from O(n) to O(log n). The map resizes (doubles capacity + rehashes everything) once `size > capacity × loadFactor` (default load factor **0.75**, so resize triggers at 12 entries for a default 16-bucket map).

Real-world Example

A **product catalog lookup** by SKU/product ID, or caching API responses keyed by request parameters — anywhere "give me the value for this key, fast" is the dominant access pattern.

Complete Code Example

```
import java.util.HashMap;
import java.util.Map;

public class HashMapDemo {
    public static void main(String[] args) {
        Map<String, Integer> inventory = new HashMap<>();

        inventory.put("Laptop", 25);
        inventory.put("Mouse", 120);
        inventory.put("Laptop", 30);    // overwrites - HashMap keys are unique

        System.out.println(inventory.get("Mouse"));                // 120
        System.out.println(inventory.getOrDefault("Keyboard", 0));  // 0

        for (Map.Entry<String, Integer> entry : inventory.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }

        inventory.merge("Mouse", 10, Integer::sum);   // Mouse = 130
        System.out.println(inventory.get("Mouse"));
    }
}
```

9. LinkedHashMap

Internal Working

`LinkedHashMap` extends HashMap and adds a doubly linked list threading through *all* entries via extra `before` / `after` references on each node, preserving either **insertion order** (default) or, if constructed with `accessOrder = true`, **access order** (most-recently-used last) — which is exactly the mechanism used to build LRU caches by overriding `removeEldestEntry()`.

Real-world Example

An **LRU (Least Recently Used) cache** — extend LinkedHashMap with accessOrder=true and override removeEldestEntry() to auto-evict the oldest entry once a size limit is hit.

Complete Code Example

```
import java.util.LinkedHashMap;
import java.util.Map;

public class LruCacheDemo extends LinkedHashMap<Integer, String> {
    private final int capacity;

    LruCacheDemo(int capacity) {
        super(16, 0.75f, true);   // true = access-order mode
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, String> eldest) {
        return size() > capacity;
    }

    public static void main(String[] args) {
        LruCacheDemo cache = new LruCacheDemo(3);
        cache.put(1, "A"); cache.put(2, "B"); cache.put(3, "C");
        cache.get(1);           // access 1 -> moves it to "most recent"
        cache.put(4, "D");      // capacity exceeded -> evicts least-recently-used (2)

        System.out.println(cache.keySet());   // [3, 1, 4]
    }
}
```

10. TreeMap

Internal Working

`TreeMap` implements `NavigableMap` using a **Red-Black Tree** — a self-balancing binary search tree where keys are kept in sorted order (natural or via Comparator). Every `put()`, `get()`, and `remove()` is **O(log n)** because it walks down the balanced tree. Unlike HashMap, iteration is always in sorted key order, and it supports navigation methods like `firstKey()`, `lastKey()`, `higherKey()`, `lowerKey()`, and range views like `headMap()` / `tailMap()` / `subMap()`.

Real-world Example

A **time-series event store** keyed by timestamp where you frequently need "all events between time A and time B" (`subMap()`) or "the next event after time X" (`higherKey()`).

Complete Code Example

```
import java.util.TreeMap;

public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap<Integer, String> events = new TreeMap<>();
        events.put(930, "Standup");
        events.put(1400, "Client call");
        events.put(1100, "Code review");

        System.out.println(events);              // sorted by key: {930=..,1100=..,1400=..}
        System.out.println(events.firstKey());     // 930
        System.out.println(events.higherKey(1000)); // 1100 - next key strictly after 1000
        System.out.println(events.subMap(1000, 1450));  // events between 1000 and 1450
    }
}
```

11. Hashtable

Internal Working

`Hashtable` is a legacy (Java 1.0) class, conceptually similar to HashMap but with **every method synchronized** (like Vector is to ArrayList). Key differences from HashMap: Hashtable does **not allow null keys or null values** (throws NullPointerException), its default capacity is 11 (not a power of 2) growing via `2n+1`, and it lacks HashMap's Java-8 treeification optimization. Modern code prefers `ConcurrentHashMap` for thread-safe maps — it offers far better concurrent throughput via lock striping / CAS operations rather than locking the entire map on every call.

Real-world Example

Mostly encountered maintaining legacy systems today; new thread-safe map needs should reach for `ConcurrentHashMap` instead.

Complete Code Example

```
import java.util.Hashtable;

public class HashtableDemo {
    public static void main(String[] args) {
        Hashtable<String, Integer> ht = new Hashtable<>();
        ht.put("A", 1);
        ht.put("B", 2);
        // ht.put(null, 5);      // NullPointerException - Hashtable disallows null keys
        // ht.put("C", null);    // NullPointerException - and null values

        System.out.println(ht);
    }
}
```

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	Unspecified	Insertion/access order	Sorted by key	Unspecified
Thread-safe	No	No	No	Yes (synchronized)
Null keys/values	1 null key, many null values	Same as HashMap	No null key (NPE)	No null key/value
get/put	O(1) average	O(1) average	O(log n)	O(1) average
Backed by	Array of buckets (+ tree bins)	HashMap + linked list	Red-Black Tree	Array of buckets

Coding — Practical Patterns

12. Frequency Counter

One of the most common interview patterns: count occurrences of each element using a `Map`.

```
import java.util.HashMap;
import java.util.Map;

public class FrequencyCounter {
    public static void main(String[] args) {
        String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};

        Map<String, Integer> freq = new HashMap<>();
        for (String word : words) {
            freq.put(word, freq.getOrDefault(word, 0) + 1);
            // Equivalent one-liner: freq.merge(word, 1, Integer::sum);
        }

        for (Map.Entry<String, Integer> entry : freq.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
        // apple -> 3, banana -> 2, cherry -> 1  (order may vary)
    }
}
```

13. Remove Duplicates

Using a `Set` to strip duplicates from a List, with two variants depending on whether order matters.

```
import java.util.*;

public class RemoveDuplicates {
    public static void main(String[] args) {
        List<Integer> nums = List.of(4, 2, 4, 5, 2, 8, 5);

        // Order doesn't matter - fastest
        Set<Integer> noDupsUnordered = new HashSet<>(nums);
        System.out.println(noDupsUnordered);

        // Preserve first-seen order
        Set<Integer> noDupsOrdered = new LinkedHashSet<>(nums);
        System.out.println(noDupsOrdered);   // [4, 2, 5, 8]

        // Sorted, unique
        Set<Integer> noDupsSorted = new TreeSet<>(nums);
        System.out.println(noDupsSorted);    // [2, 4, 5, 8]
    }
}
```

14. HashMap CRUD

```
import java.util.HashMap;
import java.util.Map;

public class HashMapCrud {
    public static void main(String[] args) {
        Map<String, Double> prices = new HashMap<>();

        // CREATE
        prices.put("Coffee", 3.50);
        prices.put("Tea", 2.75);
        prices.put("Juice", 4.00);
        System.out.println("After Create: " + prices);

        // READ
        System.out.println("Price of Tea: " + prices.get("Tea"));
        System.out.println("Has Soda? " + prices.containsKey("Soda"));

        // UPDATE
        prices.put("Coffee", 3.75);              // overwrite existing key
        prices.computeIfPresent("Tea", (k, v) -> v + 0.25);
        System.out.println("After Update: " + prices);

        // DELETE
        prices.remove("Juice");
        System.out.println("After Delete: " + prices);

        // ITERATE
        for (Map.Entry<String, Double> e : prices.entrySet()) {
            System.out.printf("%s : $%.2f%n", e.getKey(), e.getValue());
        }
    }
}
```

15. PriorityQueue in Action

A task scheduler that always processes the highest-priority task first, using a custom object + Comparator.

```
import java.util.PriorityQueue;
import java.util.Comparator;

class Task {
    String name;
    int priority;   // lower number = higher priority
    Task(String name, int priority) { this.name = name; this.priority = priority; }
    public String toString() { return name + " (P" + priority + ")"; }
}

public class TaskScheduler {
    public static void main(String[] args) {
        PriorityQueue<Task> queue = new PriorityQueue<>(
            Comparator.comparingInt(t -> t.priority)
        );

        queue.offer(new Task("Send report", 3));
        queue.offer(new Task("Fix production bug", 1));
        queue.offer(new Task("Reply to email", 5));
        queue.offer(new Task("Deploy hotfix", 1));

        while (!queue.isEmpty()) {
            System.out.println("Processing: " + queue.poll());
        }
        // Fix production bug / Deploy hotfix processed before lower-priority tasks
    }
}
```

Interview Questions (35–40)

A. Queue & PriorityQueue

1 What's the difference between add()/remove() and offer()/poll() on a Queue?

add()/remove()/element() throw an exception on failure (e.g. adding to a capacity-restricted full queue). offer()/poll()/peek() return a special value (false or null) instead — generally preferred for predictable, non-exceptional flow control.

2 How is PriorityQueue implemented internally?

As a binary min-heap stored in a resizable array. offer() appends then sifts up; poll() removes the root, moves the last element to the root, and sifts down — both O(log n).

3 Does iterating a PriorityQueue give sorted order?

No — the iterator walks the underlying heap array in no guaranteed order. Only repeatedly calling poll() returns elements in priority order.

4 How do you create a max-heap using PriorityQueue?

Pass Comparator.reverseOrder() (or a custom comparator that reverses the comparison) to the constructor, since PriorityQueue is a min-heap by default.

B. Set Interface

5 How does HashSet guarantee uniqueness?

It's backed internally by a HashMap where each element becomes a key mapped to a dummy constant value; uniqueness is enforced the same way HashMap enforces unique keys, via hashCode() and equals().

6 Why does LinkedHashSet use more memory than HashSet?

It's backed by a LinkedHashMap, where every entry carries two extra pointers (before/after) to maintain a linked list across entries for insertion-order iteration.

7 What's the time complexity difference between HashSet and TreeSet operations?

HashSet's add/remove/contains average O(1) via hashing. TreeSet's are O(log n) since it maintains a balanced Red-Black Tree for sorted order.

8 Can you add a null to a TreeSet?

Only if using a custom Comparator that explicitly handles null; with natural ordering (Comparable), adding null throws NullPointerException since compareTo() can't be called on null.

9 What happens if two unequal objects have the same hashCode() and you add both to a HashSet?

Both are stored — they land in the same bucket (a hash collision), but since equals() returns false for them, HashSet keeps both as separate entries in that bucket's chain/tree.

C. Map Interface & HashMap Internals

10 Walk through exactly what happens when you call hashMap.put(key, value).

Java computes key.hashCode(), applies HashMap's own hash() spreading function (XOR-ing the hash with itself shifted right 16 bits), then computes the bucket index as hash & (capacity - 1). If the bucket is empty, a new node is inserted. If not, it walks the chain (or tree) comparing keys with equals(); a match updates the value, no match appends a new node. If the resulting size exceeds capacity × loadFactor, the table resizes.

11 Why does HashMap use hashCode() ^ (hashCode() >>> 16) instead of raw hashCode()?

This spreads the high-order bits of the hash into the low-order bits before the bucket-index mask is applied, reducing collisions for hash codes that differ mainly in their high bits but would otherwise map to the same bucket under a simple modulo/mask on a small table.

12 Why must equals() and hashCode() be overridden together for custom keys?

The hashCode/equals contract requires that equal objects have equal hash codes. If only equals() is overridden, two "equal" keys could hash to different buckets, so the map would treat them as distinct — breaking lookups.

13 What is treeification in HashMap and when does it happen?

Since Java 8, if a single bucket's chain length reaches 8 (and the table has at least 64 buckets), that bucket converts from a linked list to a Red-Black Tree, improving worst-case lookup within that bucket from O(n) to O(log n). It converts back to a list if the count drops to 6 (untreeify threshold).

14 Is HashMap thread-safe? What happens if it's used concurrently?

No. Concurrent structural modification can cause data loss, infinite loops (in old Java 7 resize implementations), or ConcurrentModificationException during iteration. Use ConcurrentHashMap for thread-safe concurrent access.

15 Can a HashMap have a null key? How many?

Yes, exactly one null key is allowed (it's simply placed in bucket 0), and multiple null values are allowed.

16 What determines which bucket an entry goes into?

The formula hash & (table.length - 1), which works as a fast equivalent of hash % table.length only because the table length is always kept a power of two.

D. Hashing & Load Factor (Deep Dive)

17 What is the default initial capacity and load factor of a HashMap?

Default initial capacity is 16 buckets, default load factor is 0.75.

18 What exactly is "load factor" and why 0.75?

Load factor is the threshold ratio (size / capacity) at which the map resizes; a resize triggers once size exceeds capacity × loadFactor. 0.75 is a time-space trade-off: a higher factor (e.g. 0.9) saves memory but increases collision chains and slows lookups; a lower factor (e.g. 0.5) speeds lookups but wastes memory and resizes more often. 0.75 is empirically a good balance.

19 What happens during a HashMap resize?

A new array of double the capacity is allocated, and every existing entry is rehashed and redistributed into the new buckets (since the bucket-index formula depends on capacity). This is an O(n) operation, so resizes are amortized, not per-operation, costs.

20 If you know you'll insert 1000 elements, should you still use `new HashMap<>()`?

No — better to pre-size it, e.g. new HashMap<>(1334) (roughly 1000 / 0.75, rounded up to a power of two), to avoid multiple incremental resizes and rehashing passes during population.

21 What's a hash collision, and how does HashMap resolve it?

A collision is when two different keys hash to the same bucket index. HashMap resolves it via separate chaining — a linked list (or, post-treeification, a Red-Black Tree) of all entries sharing that bucket.

22 Why must table capacity always be a power of two?

It lets HashMap replace the relatively slow modulo operation (hash % capacity) with a fast bitwise AND (hash & (capacity - 1)), which is only mathematically equivalent to modulo when capacity is a power of two.

E. LinkedHashMap, TreeMap & Hashtable

23 How would you build an LRU cache using built-in Java classes?

Extend LinkedHashMap, pass accessOrder=true to the constructor (so gets move an entry to the "most recent" end), and override removeEldestEntry() to return true once size exceeds your capacity limit.

24 How does TreeMap keep its keys sorted internally?

It implements a Red-Black Tree — a self-balancing binary search tree — where every insertion/deletion triggers rotations and recoloring as needed to keep the tree height at O(log n), guaranteeing sorted in-order traversal.

25 What must a key satisfy to be usable in a TreeMap?

It must implement Comparable (for natural ordering) or the TreeMap must be constructed with an explicit Comparator — otherwise a ClassCastException is thrown on the first put().

26 What's the difference between TreeMap and HashMap in terms of guarantees?

HashMap offers O(1) average operations with no ordering guarantee; TreeMap offers O(log n) operations but guarantees sorted-key iteration and supports range/navigation queries (firstKey, higherKey, subMap, etc.).

27 Why is Hashtable considered legacy, and what should you use instead?

Every method is synchronized, causing unnecessary lock contention even without real concurrency needs, and it lacks modern optimizations like treeification. Use HashMap for single-threaded code, or ConcurrentHashMap for genuine concurrent access.

28 Does Hashtable allow null keys or values?

No — both null keys and null values throw a NullPointerException, unlike HashMap which allows one null key and multiple null values.

F. Comparisons & Practical Usage

29 How would you iterate over a Map's keys and values together?

Use map.entrySet() in a for-each loop over Map.Entry<K,V> — this is more efficient than iterating keySet() and calling get(key) for each, since it avoids a second lookup per entry.

30 What does computeIfAbsent() do and when is it useful?

It inserts a computed value only if the key is absent (or mapped to null), returning either the existing or newly computed value — commonly used to initialize a List/Set value the first time a key appears, e.g. building a Map<String, List<String>> grouping structure in one line.

31 What's the difference between Set and List regarding duplicates and order?

List allows duplicates and maintains insertion/index order; Set disallows duplicates, with ordering behavior depending on the specific implementation (none for HashSet, insertion for LinkedHashSet, sorted for TreeSet).

32 How do you find the intersection of two Sets in Java?

Copy one set, then call retainAll() with the other: `Set<T> intersection = new HashSet<>(setA); intersection.retainAll(setB);` — retainAll keeps only elements present in both.

33 How would you sort a HashMap's entries by value?

Convert entrySet() to a List, then sort with Comparator.comparing(Map.Entry::getValue), since HashMap itself doesn't support any ordering; the sorted list of entries is your output, not a re-sorted map.

34 Why can't you use a mutable object as a HashMap key safely?

If the key's fields (used in hashCode()/equals()) change after insertion, its hash code changes, so the entry effectively becomes "lost" — future lookups compute a different bucket than where it's actually stored.

35 What's the difference between Collection.remove() semantics across List, Set, and Map?

List.remove(int) removes by index, List.remove(Object) by value; Set.remove(Object) removes by value (via equals/hashCode); Map.remove(key) removes the entry matching that key, returning the previous value.

36 How would you make a HashMap thread-safe without switching classes entirely?

Wrap it with Collections.synchronizedMap(new HashMap<>()) and manually synchronize on it during iteration — though for real concurrent workloads, ConcurrentHashMap is strongly preferred for its lock-striping performance.

37 What's the output order difference you'd observe printing a HashMap vs a TreeMap of the same data?

HashMap's iteration order depends on hash bucket layout and is not guaranteed to match insertion or any logical order (and can even change across JVM versions); TreeMap always prints entries sorted by key.

38 Why is Queue's poll() generally preferred over remove() in production code?

poll() returns null on an empty queue instead of throwing, letting you handle the empty case with a simple null check rather than a try/catch, which is cheaper and reads more clearly for expected "queue is empty" conditions.

Deep Dive — Hashing & Load Factor

17. How Hashing Actually Works in HashMap

This is the single most-asked deep-dive topic in Collections interviews. Walk through it step by step.

Step-by-step put("apple", 10) on a 16-bucket HashMap

```
Step 1: h = key.hashCode()                    // "apple".hashCode() = e.g. 93029210
Step 2: h = h ^ (h >>> 16)                     // spread high bits into low bits
Step 3: index = h & (capacity - 1)             // capacity=16 -> mask = 0b1111
Step 4: if table[index] == null -> place new Node
        else -> walk chain/tree, compare via equals()
                match -> overwrite value
                no match -> append new Node (collision)
Step 5: if (++size > capacity * loadFactor) -> resize()
```

Term	Meaning
Bucket	A slot in the internal array; may hold 0, 1, or many entries (chain/tree)
Hash collision	Two different keys land in the same bucket index
Load factor	Threshold ratio (default 0.75) that triggers a resize
Capacity	Number of buckets; always a power of 2
Treeification threshold	8 entries in one bucket (with table ≥ 64) converts the bucket to a Red-Black Tree

Why load factor 0.75 specifically?

It's a deliberate trade-off documented in the JDK source itself: too high (close to 1.0) means buckets fill up before resizing, causing long chains and O(n)-ish lookups; too low (e.g. 0.25) wastes memory and forces frequent resizes. 0.75 balances average-case O(1) lookups against reasonable memory usage — good enough that the JDK team chose it as the universal default rather than exposing it as something most developers ever need to tune.

Resize Cost — Why It's Still "Amortized O(1)"

A resize doubles capacity and must rehash and relocate every existing entry — an O(n) operation. But resizes happen exponentially less often as the map grows (only when crossing each new capacity×0.75 threshold), so the total cost of all resizes across n insertions, spread over those n insertions, averages out to O(1) per insertion — the same amortized argument used for ArrayList.grow().

Illustrating Collisions Manually

```
class BadKey {
    int id;
    BadKey(int id) { this.id = id; }

    @Override
    public int hashCode() { return 1; }   // forces every key into the SAME bucket

    @Override
    public boolean equals(Object o) {
        return o instanceof BadKey && ((BadKey) o).id == this.id;
    }
}

// Even with a "bad" hashCode(), correctness is preserved (equals() still
// distinguishes keys) - but every lookup degrades toward O(n) / O(log n)
// after treeification, since all entries pile into bucket 0.
```

Coding Problems

Problem 1 — First non-repeating character in a String

```
char firstNonRepeating(String s) {
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        freq.merge(c, 1, Integer::sum);
    }
    for (Map.Entry<Character, Integer> e : freq.entrySet()) {
        if (e.getValue() == 1) return e.getKey();
    }
    return '_';   // none found
}
// firstNonRepeating("swiss") -> 'w'
```

LinkedHashMap keeps insertion order so the first single-count character found is genuinely the first in the string.

Problem 2 — Check if two Strings are anagrams using a HashMap

```
boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    Map<Character, Integer> count = new HashMap<>();
    for (char c : a.toCharArray()) count.merge(c, 1, Integer::sum);
    for (char c : b.toCharArray()) {
        count.merge(c, -1, Integer::sum);
        if (count.get(c) == 0) count.remove(c);
    }
    return count.isEmpty();
}
// isAnagram("listen", "silent") -> true
```

Problem 3 — Find the top K frequent elements using PriorityQueue

```
List<Integer> topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    PriorityQueue<Integer> minHeap =
        new PriorityQueue<>(Comparator.comparingInt(freq::get));

    for (int key : freq.keySet()) {
        minHeap.offer(key);
        if (minHeap.size() > k) minHeap.poll();   // evict lowest-frequency
    }

    List<Integer> result = new ArrayList<>(minHeap);
    Collections.reverse(result);
    return result;
}
// topKFrequent([1,1,1,2,2,3], 2) -> [1, 2]
```

Classic combination of frequency-counter (Map) + PriorityQueue-as-a-bounded-heap pattern.

Problem 4 — Group anagrams together

```
List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String w : words) {
        char[] chars = w.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);          // sorted string = anagram signature
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}
// ["eat","tea","tan","ate","nat","bat"] -> [[eat,tea,ate],[tan,nat],[bat]]
```

Problem 5 — Detect a cycle using a HashSet (visited-node tracking)

```
boolean hasDuplicateWithinDistance(int[] nums, int k) {
    Set<Integer> window = new HashSet<>();
    for (int i = 0; i < nums.length; i++) {
        if (window.contains(nums[i])) return true;
        window.add(nums[i]);
        if (window.size() > k) {
            window.remove(nums[i - k]);
        }
    }
    return false;
}
// Checks if any value repeats within a sliding window of size k - a
// sliding-window + HashSet pattern that appears constantly in interviews.
```

Problem 6 — Merge K sorted lists' smallest elements using PriorityQueue

```
List<Integer> mergeSmallestFromEach(List<List<Integer>> lists) {
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
    // a[0] = value, a[1] = which list, a[2] = index within that list
    for (int i = 0; i < lists.size(); i++) {
        if (!lists.get(i).isEmpty()) {
            pq.offer(new int[]{lists.get(i).get(0), i, 0});
        }
    }

    List<Integer> result = new ArrayList<>();
    while (!pq.isEmpty()) {
        int[] top = pq.poll();
        result.add(top[0]);
        List<Integer> list = lists.get(top[1]);
        int nextIdx = top[2] + 1;
        if (nextIdx < list.size()) {
            pq.offer(new int[]{list.get(nextIdx), top[1], nextIdx});
        }
    }
    return result;   // fully merged, sorted sequence
}
```

The k-way merge pattern — foundational for external sorting and merging log streams.

AI-Style Interview Questions

Scenario-based reasoning questions in the style modern AI-driven interview screens tend to ask — testing judgment and trade-offs, not just recall.

Your HashMap-based cache is showing degraded lookup performance in production after months of uptime, even though total entries stayed roughly constant. What would you investigate?

Likely a poor hashCode() implementation on the key class causing many keys to collide into the same buckets — check for a constant or low-entropy hashCode(). Also check if keys are mutable and being changed after insertion, which can "lose" entries and cause inconsistent behavior. Treeification would partially mask this (O(log n) instead of O(n)) but wouldn't fully fix it — the real fix is a better hashCode() distribution.

You need to track the 5 most recent unique searches a user made, most-recent-first, no duplicates. Which collection(s) would you combine?

A LinkedHashSet with accessOrder-like re-insertion logic, or more directly, remove-then-re-add the search term on every repeat search so it moves to the end/front, combined with a manual cap check to evict the oldest once size exceeds 5 — the same underlying idea as the LinkedHashMap LRU pattern, applied to a Set.

A teammate proposes using TreeMap everywhere "because it's sorted and safer." How do you respond?

Push back gently — TreeMap's O(log n) operations are slower than HashMap's O(1) average for the common case of simple key lookups, and "sorted" is only valuable if you actually need ordered iteration or range queries. Recommend HashMap by default, reaching for TreeMap specifically when sorted traversal or navigation methods (firstKey, subMap, ceiling) are actual requirements.

You're asked to design a system that processes customer support tickets by priority, but tickets can also be canceled (removed) before being handled. Which data structure fits, and what's the catch?

PriorityQueue fits the priority-based processing, but it lacks efficient arbitrary removal — remove(Object) is O(n) since it must linear-scan the heap array to find the element. For frequent cancellations at scale, consider pairing the PriorityQueue with a "lazy deletion" HashSet of canceled IDs (skip them on poll) or use an indexed heap structure.

Explain, as if to a new grad, why HashMap capacity is always a power of two.

It lets Java compute the bucket index with a fast bitwise AND (hash & (capacity-1)) instead of a slower modulo operation, and this bitwise trick is only mathematically equivalent to hash % capacity when capacity is a power of two — so it's a deliberate performance optimization baked into the data structure's design.

Your team is debating HashMap vs ConcurrentHashMap for a read-heavy, occasionally-written shared cache accessed by 50 threads. What do you recommend and why?

ConcurrentHashMap — plain HashMap isn't thread-safe and risks corruption or infinite loops under concurrent structural modification. ConcurrentHashMap uses lock striping (and CAS operations for many paths), so concurrent reads proceed largely without blocking and writes only lock a small segment, giving far better throughput than externally synchronizing a plain HashMap.

Mock Interview Transcript

A realistic back-and-forth covering today's topics, as it might unfold in a real technical round.

INTERVIEWER

Let's dig into HashMap. Can you walk me through what happens internally when I call put("key", "value")?

CANDIDATE

Sure. Java first computes the key's hashCode, then HashMap applies its own spreading function — XOR-ing that hash with itself shifted right 16 bits — to spread high bits into the low bits. It then computes the bucket index using hash AND (capacity minus 1), which works like a fast modulo since capacity is always a power of two. If that bucket is empty, it places a new node there; if not, it walks the existing chain, using equals to check for a matching key to update, or appends a new node if it's a genuine collision.

INTERVIEWER

What happens if a lot of keys collide into the same bucket?

CANDIDATE

Since Java 8, if a single bucket's chain grows to 8 or more entries — and the table itself has at least 64 buckets — that bucket converts from a linked list into a Red-Black Tree, which improves worst-case lookup within that bucket from linear time to logarithmic time.

INTERVIEWER

What triggers a resize, and why is the default load factor 0.75?

CANDIDATE

A resize happens once the number of entries exceeds capacity times load factor — so for the default 16-bucket map, that's after the 12th entry. 0.75 balances memory usage against collision rate: a higher factor means buckets fill up more before resizing, leading to longer chains and slower lookups, while a lower factor wastes memory and forces more frequent resizes. 0.75 is the JDK's chosen middle ground.

INTERVIEWER

If I need my Map to stay sorted by key at all times, what would you use instead?

CANDIDATE

TreeMap — it's backed by a Red-Black Tree, so keys are always kept in sorted order, and you get navigation methods like firstKey, higherKey, and subMap. The trade-off is that operations become O(log n) instead of HashMap's average O(1), so I'd only reach for it if I actually need that ordering or range-query behavior.

INTERVIEWER

How would you build a simple LRU cache using built-in Java classes?

CANDIDATE

Extend LinkedHashMap, construct it with accessOrder set to true so that every get() moves an entry to the most-recently-used end, and override removeEldestEntry() to return true once the map's size exceeds my chosen capacity — LinkedHashMap will then auto-evict the least-recently-used entry on the next put.

INTERVIEWER

Suppose I have a HashSet of custom objects and two "equal" objects still both got added. What went wrong?

CANDIDATE

Most likely the class overrode equals() but not hashCode(), or hashCode() is inconsistent with equals(). Since HashSet is backed by a HashMap, two objects that are equal by equals() but produce different hash codes can end up in different buckets, so the set treats them as distinct entries.

INTERVIEWER

Last one — when would you choose a PriorityQueue over a sorted List?

CANDIDATE

When I mainly need repeated access to the current minimum or maximum and don't care about the full sorted order at any given moment — PriorityQueue gives O(log n) insert and removal of the extreme element, whereas keeping a List fully sorted after every insertion would cost O(n) per insert due to shifting.

INTERVIEWER

Solid understanding across the board — that wraps up today's round.

CANDIDATE

Thanks — happy to go deeper into ConcurrentHashMap's internals or Red-Black Tree rotations if that would help.

— End of Day 2 Notes · Java Collections Framework · Revise, Code, Repeat —